ソフトウェアデザイン 2026

教師用補足・公開解説

第2〜15回

情報システム学科 3年生　選択2単位

授業で案内する提出先・期限に従って使用する。

演習のフォルダ構成は本教材の推奨例。

第02回 教員用補足：Pythonを自分で確かめる

学生用本編は `lectures/02.md`、自習演習は `exercises/02.md`。自習演習は、各例の予測・実行・理由の確認を単独で進められる構成です。確認問題の理由つき解答を学生版にも載せています。この資料は、提出運用と理解の観察、支援の順序を扱います。正式な授業時間や点数配分は設定しません。

学習の進め方

まず、Python 3.12以上を使っていることと、自分のファイルを保存して実行できることを確認します。コード・出力・ターミナルのコマンドの区別ができていない段階では、構文の説明を増やす前に、短いprintの例を保存・実行する流れへ戻します。

学生には、実行前の予測を1行でも残すよう促します。予測の正誤だけで評価せず、実際の結果との違いを説明し直せるかを観察します。解答を先に読んだ学生には、値や条件を1つだけ変えた入力で、再度予測してもらいます。

全ての項目を同じ速度で進める必要はありません。値・代入・分岐・反復・関数の基礎を確認してから、共有とコピー、入力検証へ進めます。自習で止まった場所は、節番号・入力・予測・結果を示せると、支援しやすくなります。

概念ごとの観察と支援

概念	観察する説明	誤解があったときの支援
値と型	整数200と文字列"200"を区別する	同じ表示でも加算・連結が違う例を実行
代入	右辺を評価して名前を結び直す	priceとoriginalが何を指すか書く
演算・比較	優先順位、= と == を区別する	括弧を付けた式と条件式を比較
if	字下げが実行範囲を決める	quantityを0と2に変えて分岐を追う
for	totalが処理済み要素の合計になる	反復ごとの加算前後を表にする
while	条件を再評価し、終了へ近づく	countの更新と条件の対応を示す
list	0始まり、要素数と末尾位置を区別する	3要素の位置を0、1、2で書く
共有・コピー	結び直しとオブジェクト変更を区別する	aliasとcopyの例を別々に実行
dict	キー存在と既定値の意味を説明する	不足quantityを0で受け入れる危険を考える
関数	引数、戻り値、終了を区別する	print版とreturn版の呼び出し元を比較
スコープ	ローカル名の変更と共有入力の変更を分ける	整数引数とリスト引数を比較
エラー	種類・行・その時点の値を確認する	1つのエラー例だけを実行して追う

この表は理解の観察用であり、正式な採点表ではありません。

確認問題の補足

学生用の解答に加えて、次を押さえます。

- 代入は、後の更新に追従する式を保存する操作ではありません。整数を再代入する例と、同じリストの要素を変える例を混ぜないで説明します。
- 型変換は元の値を必ず書き換える処理ではありません。`count = int(text)` ではtextはstrのままです。`input()` の戻り値もstrです。

- `/` の整数同士の結果もfloat。`//` は床除算で、負数に対して常に0方向へ切り捨てるわけではありません。基本例は正の整数とし、負数は補足にします。
- `and`・`or` は短絡評価します。一般にはオペランドを返すので、常にTrue/Falseだけを返すと説明しないようにします。学生例は比較式を組み合わせているのでboolになります。
- `range(3)` は0、1、2の3回。forのループ対象はリストだけではありませんが、この回は要素を順に取り出す例から確認します。
- 空入力の合計0は、初期値と0回の反復から説明できます。単に「空は特別扱い」と暗記させる必要はありません。
- `copy()` は外側のリストを作り、参照先まで複製しません。入れ子の辞書を変更する例で、何が共有されているかを確認します。
- returnなしの関数もNoneを返します。「戻り値が全く存在しない」とは言いません。returnを実行したら、その呼び出しの後続行へ進みません。
- ローカルな名前への再代入は外側の同名変数を変更しません。ただし、関数へ渡したリストや辞書自体の変更は、呼び出し元へ見えます。
- 型注釈だけでは入力を拒否しません。利用者の入力検証をassertだけに任せず、ifとraiseを使います。

エラーの学習支援

最初は、エラーを消すことより、何を確認する必要があるかを言えることを重視します。NameErrorなら名前の定義・綴り、TypeErrorなら演算する型、ValueErrorなら許可する値、IndexErrorなら位置、KeyErrorなら必須キーに戻ります。

構文エラーはコードの解釈段階で止まり、実行中の例外は到達した処理で発生します。字下げを変えると実行範囲が変わる点も、単なる見た目として扱わないようにします。エラー文の正確な全文や矢印表示はPythonの版で差があるため、種類と原因の説明を中心に確認します。

意図したValueErrorで停止する例は、正常終了の例と分けて示します。CLIのエラー表示を、そのまま本人の失敗とみなさないようにします。無限whileを試してしまった場合は、Control+Cで止め、終了条件と更新へ戻します。

補助デモの使い方

`python3 examples/02-basics/concepts.py CASE` で概念ごとの例を実行できます。ケース名は学生資料の表に示しています。CASE未指定・未知のCASEはCLIの使い方のエラーとして終了コード2。各デモの正常終了は0です。exceptionsは意図した例外を内部で捕捉するため、例外を表示してもCLIとしては0で終了します。

tracebackの表示は実際の例外から得た抜粋であり、絶対パス・行番号を省略しています。学生が自分のコードのエラーを読む際は、通常のtracebackのファイル名・行番号へ戻ります。デモの抜粋を完全なエラー全文だと扱わないようにします。

学生本文の短例とデモでは、入力や表示の位置が異なります。例えば本文のwhileは更新前のcountを表示し、while-updateは更新前後を1行に表示します。結果が違って見える場合は、同じ仕組みのどの時点を表示したかを比べます。typesの0.1+0.2などは発展の観察とし、注文の円単位の整数仕様と分けます。

注文集計の契約

学生課題の `summary(orders) -> int` は、注文辞書のリストを受け取り、円単位の総額を整数で返します。公開例の `summarize` は品目別数量を含む辞書を返す別の関数なので、課題の戻り値と混同しないよう説明します。

必須条件は次のとおりです。

- 入力全体はlist、各要素はdict。
- name、price、quantityを持ち、nameはstr。空文字も受け入れる現行契約。
- priceとquantityは0以上の厳密なint。bool、float、strを拒否。
- 空入力の総額は0。数量0・単価0は有効。
- キー不足、型違い、負数は理由が分かるValueError。
- 入力のlist・dictを変更しない。重複商品名の各行も加算する。

boolはintの派生なので、`isinstance(value, int)` だけでは契約を満たしません。厳密型の判定は `type(value) is int` などで行います。型を確認してから負数を比較すれば、文字列の比較によるTypeErrorを避けられます。検証方法は複数の書き方を認めますが、契約の例外型を揃えます。

解説例と検証の目安

既定の公開デモはノート200円×2とペン120円×3、総額760円。`summary([])` は0。数量0ではその行の小計0。商品名が同じでも、各行の価格で小計を計算します。

正常・空・数量0・負数・数の文字列・小数・bool・キー不足・入力全体の型違い・要素の型違いを確認します。テストで期待値を同じ実装ロジックから計算し直すと、同じ誤りを見逃しやすいので、小さい値を手で求めます。

入力非変更は、入れ子のdictを含むコピーと比較します。`before = orders` は同じオブジェクトなので不適切。`orders.copy()` も内部dictの変更を見逃します。検証にはdeepcopyを使えますが、集計関数本体に入力のコピーを必須とするわけではありません。入力を読むだけで合計を作れば非変更を保てます。

戻り値の型を観察するときは、760.0やTrueを整数と等値比較するだけで通していないかにも注意します。今回の戻り値はintです。

口頭確認の問い

本人のコードで具体的な入力を指し、次を確認します。

1. 2件目を処理する前後で、totalは何から何へ変わりますか。
2. returnをforの内側に置くと、何件まで処理しますか。
3. 空入力ではどの行を実行しますか。
4. quantityが0、True、"2"なら、それぞれどこを通りますか。
5. 入力のdictを変えていない根拠はどこですか。
6. AIの提案を採用した箇所を、どの入力で確認しましたか。

「わかりましたか」という問いだけで理解を判断しません。説明に詰まった場合は、入力を1件へ減らす、状態を表にする、表示する値の型を確認するといった支援を行います。

提出運用

正式な提出先・期限・点数配分・参照条件は、実授業で案内します。過年度の規則を現年度の運用と推定しません。学生本文の提出案内は末尾に短くまとめ、授業の中心を学習内容に置きます。

推奨成果物は `submission/02/summary.py`、`test_summary.py`、短いREADME（実行方法）とreflection（予測・修正・AI支援）。標準unittestで正常・境界・異常入力と入力非変更を確認できます。`if __name__ == "__main__":` を用いる場合、その仕組みはコードのimport時と直実行時の違いを短く補足し、計算関数の理解より先行させません。

提出前には実行方法と必要ファイルを確認し、提出後は届いたことを本人が確認できる案内を用意します。未完成の場合も、動く範囲と再現入力を記録するよう促します。正式な再提出の扱いは別途案内します。

AI利用の有無だけで理解を判定しません。使用の用途、提案の採用・修正範囲、本人が予測・実行・説明した内容を観察します。実際の学生の名前・コード・点数・個別講評はこの公開資料へ記載しません。

第3回　教師用補足

AIの利用を推奨し、答えの取得だけでなく学習の変化を観察します。特定サービスの契約・UIを必須にしません。環境が用意できない学生には、自作の提案例と相互質問で検証活動を継続します。

確認問題の解説

1. 出力は仕様を取り違えていても自然に読める。実行例と公式API仕様で確認する。
2. 閉区間で候補1個が残るleft==rightを検査しないため、while left<rightはこの実装では誤り。半開区間では別の境界定義が必要。
3. 仕様から手計算する、単純な線形探索と照合する、境界ごとに期待値を独立に作る。生成したテストの通過だけで全範囲の正しさは証明しない。
4. 提案の要点、却下の理由、反例・前提・仕様の差を残す。

授業中の観察

学生に未使用の1要素・2要素入力を渡し、探索のleft/rightを説明してもらいます。修正版を見て理解した学生も、その後に別の反例や変種を説明できれば学習を確認できます。正式評価点はシラバスにないため、本補足では点を割り振りません。

過去教材との対比

2025のPython基礎は複数回にまたがり、AI学習回は独立していませんでした。2026正本に従い、第2回のコードを材料としてAI学習・検証・振り返りを独立した第3回に配置しました。過去の成績割合や提出条件は継承していません。

公開教材として自作した例のみを扱い、実際の学生回答・AIチャット・採点情報を載せません。

第04回 教員用：データ構造とアルゴリズム

学生配布用スライドは `lectures/04.md`、演習は `exercises/04.md` です。ここは解答と学習支援用です。正式な授業時間・課題の点数配分を設定する資料ではありません。進行は受講者の状態に合わせ、概念説明、実演、段階演習、確認を組み合わせます。

この回の位置付け

2026年の計画に従い、メモリサイズ、計算量、キュー、スタック、ソート、線形探索、全探索、二分探索、DFS、BFS、木を1回で概観します。すべてを同じ深さで実装させるのではなく、処理順の予測、二分探索の前提、操作数の増え方を到達の中心にします。

前回のAI支援による開発へ接続し、「提案されたコードが正しい条件」と「入力が増えたときの費用」を自分で確認させます。

学生スライドの「今日、深く追うところ」を先に示します。処理順の予測と二分探索の状態追跡を中心にし、ソートの方式、計算量のケース、メモリの内部実装を同じ深さの白紙実装にしません。演習の段階1〜3と提出物は、学生用演習に記載したとおりとします。補足のNode・yieldの説明を、新しい実装課題や提出要件にしません。

実演時の注意

Python 3.12以上を事前確認します。メモリのbyte数は環境差があるため、講師の出力への一致を正解条件にしません。`sys.getsizeof` は直接のオブジェクトの大きさを返し、参照先の全体を再帰的に数えません。浅いコピーは要素への参照を複製するため、要素の実体をすべて新しくする処理ではありません。

CPythonのlistは可変長配列として扱います。連結リストと説明しません。dequeの両端操作とリスト先頭削除の違いは、要素をずらす必要性で説明します。appendは、リストの容量拡張があるため、ならしのO(1)と瞬間の処理費用を区別します。

スライドの二分探索は半開区間、公開例の二分探索は閉区間を使います。どちらも正しいですが、境界の更新方法を混ぜません。不在の戻り値は両方Noneで揃えます。

二分探索の状態表の使い方

まず半開区間の一例を最後まで追い、その後で閉区間と比べます。入力は両方 `[2,4,6,8]`、探索値は5です。半開区間は `(lo,hi,mid) = (0,4,2) → (0,2,1) → (2,2,候補なし)` です。閉

区間は `(low,high,middle) = (0,3,1) → (2,3,2) → (2,1,候補なし)` です。中央値を比べる順序は違いますが、不一致と確認した位置を除き、あり得る候補を残す点は同じです。

候補1個の `[6]` を使い、半開区間 `(0,1)` と閉区間 `(0,0)` がともに比較を行うことを確認します。6なら0を返します。5なら半開区間 `(0,0)`、閉区間 `(0,-1)` となりNoneを返します。空列は最初からこれらの候補なしの状態になります。while条件の暗記ではなく、「右端を含むか」「候補が何個か」を説明できるかを見ます。

DFSの表現をつなぐ

学生は第2回のリスト・関数・returnから読み始められます。訪問順の表を追ってから、辞書版の `dfs_names` を読みます。内側の `visit` は各呼出しで新しい出力リストを作らず、外側が用意した同じ `order` に追記します。訪問後に呼出し元のforへ戻る場所を指で追います。内側の関数を定義するだけでは木を訪問せず、`visit(root)` から実行が始まります。

公開 `trees.py` では `node.value` が名前、`node.children` が子のリストです。辞書から情報を引く書き方と節点の情報を読む書き方の差を示し、どちらもＡＢＤＥＣＦをリストで返すことへ戻します。Nodeのクラス定義・dataclassそのものの習得を到達条件に加えません。公開版のDFSは `root is None` なら空リストを返します。

`yield` の補足では、関数を呼んだだけで全訪問が完了するわけではなく、値を要求するたびに先へ進むことを説明します。`yield from` は子のジェネレーターの値を順に渡します。`list(dfs_stream("A", children))` で取り出した順序は同じです。公開版は最後にリストを返すため、ジェネレーターの白紙実装を必須にしません。

DFS/BFSの補助記憶の比較は、公開版で訪問結果のリストを除く条件を明示します。結果を全件保存する領域はどちらもO(V)で、再帰呼出しの深さとキューの幅とは別に数えます。

時間は多数の要因で変動します。計算量を推測する主教材は、小さい入力の操作数モデルにします。`memory_complexity.py` の回数は式から計算したモデル値であり、実行した比較を逐一計測した値ではありません。巨大な二重ループの結果をメモリに蓄積する演習は行いません。

確認問題の解説

1. 二分探索は、中央との大小関係から半分を除外する。未整列では、目的の値が除外側に存在し得る。例 `[8,2,6,4]` で8を探すと、昇順の仮定が成立せず見落とせる。

2. n件から2つの位置を `i < j` で選ぶ候補数は n(n−1)/2、増え方はO(n²)。同じ位置を許す二重ループならn²。全探索一般が常にO(n)またはO(n²)とは限らず、候補集合次第。
3. リスト末尾のpopを使えばLIFOとなり、DFSに近い処理順になる。BFSの距離順処理を保証するFIFOが崩れる。
4. list本体には管理情報と要素への参照がある。getsizeof(list)だけでは、別オブジェクトである各要素の全量を説明できない。

演習の答えと支援

- スタックC B A、キューA B C。
- 自作木のDFSはA B D E C F、BFSはA B C D E F。
- 線形探索 `[8,2,6,4]` の6はインデックス2。整列済みコピー `[2,4,6,8]` の6も2だが、同じ数になったのはこの入力による。別の値で元の位置と整列後の位置が異なることを示す。
- 二分探索の区間 `[lo,hi)` は、答えがあればその区間内にあるという条件を保ちつつ縮小する。空列はwhileに入らず `None`。
- n(n−1)/2の候補数は、n=3,4,5で3,6,10。

初学者には、比較した位置とlo/hiを紙で追わせます。理解が進んだ学生には、重複の先頭を返す版や、ソートを含む複数検索の総コストを検討させます。

よくある誤解

「DFS/BFSには整列が必要」は誤りです。列の二分探索と区別します。隣接順や子の順は訪問順を変えますが、整列が探索の成立条件ではありません。通常の全走査はO(V+E)、木ではO(V)です。再帰DFSの呼出し領域は高さに依存します。BFSのキューは幅に依存します。

「O記法が秒数そのもの」、「動的メモリ管理はすべてGCと同義」、「組込みソートの1回の実測でO(n²)が証明できる」といった説明を避けます。

口頭確認と到達の目安

「未整列で二分探索を使える理由はありますか」「区間が1要素でも終了しますか」「BFSのキューにはどの深さの頂点が残っていますか」と、入力に戻れる問いを使います。

到達の目安は、構造の処理順を予測できること、二分探索の前提と終了条件を説明できること、操作数とメモリの測定対象を区別できることです。誤りがあっても、再現入力から説明を修正できるかを確認します。正式な評価配点は担当教員が別途案内します。

第05回 教員用：グラフの設計

学生配布用は `lectures/05.md` と `exercises/05.md` です。解答や説明例を含む教員用資料です。正式な時間・採点配分は設定しません。

到達目標と進行

頂点・辺・向き・重みを要求から決め、BFSの発見時記録と親の復元へつなぎます。グラフ理論の網羅より、次回の経路課題の仕様を説明できることを重視します。実例の駅名や路線図は転載せず、架空校内マップでモデルの範囲を話します。

説明、図と隣接リストの照合、手作業のキュー追跡、入力変更、確認問題の順で進めます。正式な所要分数を固定しません。

実演の正解

`examples/05-graphs/graphs.py` のグラフはA-B、A-C、B-D、C-D、孤立Xです。AからDのBFS経路はA B Dです。Aから到達する集合はA B C D、連結成分は `{A,B,C,D}` と `{X}` の2つです。

Aの隣接順をB、CとしたBFSの追跡例：開始キューA、Aの取り出し後B C、Bの取り出し後C D、Cの取り出し後D。Dの親はBのままです。探索をgoal発見時に止める実装では、最後までの訪問順を求める実演と出力が異なるので、停止条件を確認します。

次数の説明

今回の単純無向グラフでは次数は隣接頂点数、Aは2、Xは0です。無向の自己ループを扱う拡張では次数の数え方が変わるので、今回の前提を示します。有向グラフは入次数・出次数と呼びます。値の符号を意味しません。相互に有向辺が存在することと、モデル全体を無向と定義することも区別します。

確認問題の解説

1. 一方通行を無向として登録すると、許可されない逆向き移動を返す。元の意味を変える誤モデル。
2. 頂点をキューに追加した時点で発見済みにする。取り出した時点まで遅らせると、複数の親から同じ頂点を重複追加し得る。

3. 重みなし、または全辺の移動費用が同じ正の定数の場合、最少辺数と最少総費用が一致する。時間や料金が辺ごとに異なれば、BFSだけでは最少総費用を保証しない。
4. 関数は既知の孤立頂点への到達不能を `None` で返せる。未知の頂点名は別に検証し、入力エラーとして扱う。

演習の解説

無向辺B-Dを両方向から削れば、A Dの最短経路はA C Dです。さらにC-Dを両方向から削れば到達不能です。片側だけ削ると「無向」の前提を壊します。有向A B、B DではAからDは到達可能、DからAは到達不能です。出次数は隣接リスト長、入次数は全ての隣接リストに対象が現れる回数で求めます。

同じ長さの最短経路が複数ある場合、1本返す仕様なら、経路文字列の完全一致だけを正解条件にしません。開始・終了、各辺の存在、最少辺数を確認します。教材の固定データは順序を固定してA B Dを期待します。

よくある誤解と支援

訪問済みは循環対策だけでなく、重複した発見を避けます。parentを後から上書きすると、最初の距離の根拠を失います。DFSで経路が見つかることと、その経路が最短であることを区別します。全経路の列挙はこの回のBFS1本返却とは別の仕様であり、候補数が非常に増え得ます。

無向グラフの循環検出は直前の親への辺を除外します。単純なvisited判定だけで親への戻りを循環と誤判定しません。有向グラフで到達可能な集合を1つ求めても、強連結の証明にはなりません。

理解に詰まった学生には、コードをいったん閉じ、図の辺を両方向の隣接リストへ転記させます。次にキューの先頭を1個ずつ消し、追加した親だけを書きます。

口頭確認と到達の目安

「この辺の意味は何ですか」「Dの親がCへ変わらない理由は何ですか」「時間の最短へ仕様を変えると、何のデータが必要ですか」と確認します。

到達の目安は、モデルの前提を言えること、対称な隣接リストを作れること、発見時記録と復元を説明できることです。正式な評価は別途案内します。

第06回 教員用：実装と授業内提出

学生版は `exercises/06.md`、解説用の参考実装は `examples/06-route-project/route_reference.py` です。学生用スターターは `route_starter.py` です。公開参考実装と学生課題を区別し、参照可否・提出方法・締切は実際の授業で案内します。過年度の提出先、点数配分、協働に関する規則をこの資料から推定しません。

授業の目的と進行

データ構造とアルゴリズムの知識を実装で確認し、授業内に提出します。実装だけで終了せず、正常・境界入力、成果物整理、提出完了の確認まで扱います。正式な授業時間は未設定です。進行の状況を見て、状態追跡の支援と実装支援を分けます。

課題は無向・重みなしグラフで最少辺数の経路1本を返すものです。全経路の列挙や重み付き最短経路は必須要件にしません。

開始確認の後は実装・検証を主活動にします。全ページを先に説明してから作業を始める進行は避け、段階2〜5を作業中の参照にします。環境・コピー先の問題はBFSの問題と分けて支援します。検証、成果物整理、提出完了確認の枠を先に取り、作業が遅れた学生にも動く範囲と再現入力を残させます。この進め方は作業の順序を示すもので、課題仕様と必須ケースは学生用演習のとおりとします。

APIとCLIの契約

`route_data.py` は `GRAPH` と `STATIONS` を提供します。`find_route(graph, start, goal)` は経路のlist、到達不能時はNoneを返します。未知の駅名や無向辺の不整合はValueErrorです。入力検証をBFSの前に実行します。CLIはSTART GOAL、省略時A D、失敗は終了コード2です。

学生版は同駅と隣駅のみ回答するため、初期のA D失敗は意図した未実装です。環境エラーと混同しません。作業先へ学生版とデータを同じディレクトリにコピーします。データのimport失敗なら、先にコピー先と実行パスを確認します。Python 3.12以上で検証します。

参考実装の説明

開始点をdequeに入れ、parentにstart:Noneを置きます。FIFOで取り出し、未発見の隣を発見した時点でparentを設定して追加します。goalが発見済みなら、goalから親のNoneまでたどって経路を反転します。到達不能なら復元前にNoneを返します。

親は単に直前の頂点の候補ではなく、最初の発見に使った頂点です。1回だけ登録することで重複を避け、最少辺数の層の根拠を保ちます。時間は隣接リストと平均的な辞書操作のモデルでO(V+E)、追加領域O(V)です。入力の検証もグラフの頂点・辺を調べます。

必須ケースの正解

- A DはA B D、移動数2。
- D AはD B A、移動数2。
- A AはA、移動数0。X XもX、移動数0。
- A BはA B、移動数1。
- A Xは関数でNone、CLIで到達不能と終了コード2。
- UNKNOWN Dは関数でValueError、CLIで入力エラーと終了コード2。

経路は頂点列なので辺数はlen(path)-1です。経路が複数ある自作グラフは、各辺と最少辺数を確認し、1つの文字列に限定しません。

確認問題の解説

1. 開始点の親をNoneにすると、開始点までたどったときの停止条件になる。同時に、開始点を発見済みにする。
2. popはLIFOで奥の枝を先にたどる。FIFOによる距離の層の順序を保てず、最少辺数を保証できなくなる。
3. 同じ出発点と到着点ではpathの長さ1、辺数0。孤立駅でも同駅の移動は0。
4. 未到達のgoalはparentにないため、復元を始めればKeyErrorなどになる。到達不能の仕様に合う値を先に返す。

支援と理解の確認

無限ループなら、未発見判定と登録の時点を確認します。逆向きの経路なら復元後の反転、長い経路ならFIFOとparentの上書きを見ます。実装を丸ごと渡す前に、A Dでキューとparentの変化を学生本人に記録させます。

AI利用の有無から理解を断定しません。「提案をどこで確かめたか」「この入力でどの行が実行されるか」と確認します。支援が多かった学生には、同駅・孤立駅の説明など小さい範囲から確認します。

提出確認と到達の目安

推奨成果物は実装・必要データ・検証コード・README・test-results・reflectionです。提出先に成果物が届いたことを学生本人に確認させます。未完成の場合も、実行できる範囲、再現入力、残る課題の記録を促します。

到達の目安は、仕様どおりの戻り値、BFSと復元の説明、境界入力の検証、AIや参考実装の参照範囲の記録です。これは学習支援の基準で、正式な配点ではありません。個別の実採点情報はこの公開資料へ記載しません。

第07回 教員用：講評と質疑

学生版は `lectures/07.md` と `exercises/07.md` です。共通講評は全て自作の架空提出例です。実学生のコード、氏名、学籍番号、点数は含めません。実際の個別確認は、提出者と担当教員の間で提出内容の理解を扱います。

進行と目的

共通例の仕様違反を反例で説明し、その後に本人の提出物の質疑を行います。共通講評だけで終了させず、各人の設計・検証・AI支援範囲の説明へつなぎます。時間配分は授業状況に合わせ、正式な分数をこの教材では設定しません。再提出の有無や方法は実授業で案内します。

授業前に履修人数と個別確認の時間を照合します。教員1人・確認枠T分・学生N人なら一人当たりT/N分です。全ての問いを各人へ同じ深さで尋ねる前提を置かず、本人のコードと検証から焦点を選びます。確認待ちの学生はペアで状態追跡と反例検証を進めます。ペア活動を教員による本人確認の代替と自動的に扱わず、確認順と方法を授業で案内します。実際の確認記録は公開教材に保存しません。

架空例の正解

この回のグラフは第06回と別物です。辺はA-B、B-E、E-F、F-D、A-C、C-D、Xは孤立です。`flawed_route.py` の訪問済み付きDFSは、最初にA B E F Dを返し、移動数4を最短と表示します。経路そのものは妥当ですが、最少性は誤りです。`improved_route.py` のBFSはA C D、移動数2を返します。

A-Cを両方向から削除すれば、4辺のA B E F Dが最短になります。しかし特定の入力で最短が返ることと、全入力の仕様を満たすことは別です。visitedは終了性・重複回避に寄与しますが、DFSの最短保証を生みません。

確認問題の解説

1. visitedがあってもDFSは最短を保証しない。隣接順B Cなら、B側の深い枝を先にたどってDを見つける。
2. 妥当な経路は、始点終点と各辺の存在を満たす。最短経路には、もっと少ない辺の経路がないという追加の条件がある。

3. DFSが先に深い経路を見つける一方、別の分岐に短い経路がある小さい入力を使う。同駅や隣駅だけでは違いを検出しにくい。
4. 修正前の反例を再実行し、経路の妥当性と最少辺数を確認する。到達不能、同駅、未知入力も再確認し、新しい不具合が生じていないかを見る。

共通講評の例

「A Dの出力は実在する経路ですが、4辺です。このグラフにはA C Dの2辺があるため、最少辺数の条件を満たしていません。FIFOと最初の親記録を使う版を検討し、同じ入力とA Xで再確認してください。」

これは教材用の例文です。実学生の出来や能力を評価した発言ではありません。改善の対象・証拠・次の確認を含めます。

個別の質疑の問い

- この提出物の最短の定義は何ですか。
- A Dの探索で、2回目の取り出し後のキューと親辞書を示せますか。
- parentへの登録を取り出し後まで遅らせると、何が変わりますか。
- 孤立駅と未知駅で、どの処理と戻り値が違いますか。
- 自作テストは、どの誤りを見つける狙いですか。
- AIまたは参考版を使った箇所を、自分の入力でどう確認しましたか。

本人の第06回グラフと、共通講評の第07回グラフを混ぜません。第06回でA DはA B D、第07回でA DはA C Dです。

説明に詰まった場合の支援

すぐに解答を代読せず、同駅、隣駅、2段経路へ入力を小さくします。キューから消す要素、追加する要素、parentへ初めて記す組を紙に書きます。本人が結果を予測してから実行し、説明を修正する機会を作ります。

AIへの質問の詳細な履歴を公開資料へ転載する必要はありません。支援の用途、採用範囲、本人による検証の要約を確認します。AIを未使用でも、そのことだけを理解の証拠としません。

到達の目安

仕様と挙動の対応を説明できること、小さい反例で改善点を示せること、本人の提出物の主要状態を追えること、修正の証拠を残せることを確認します。正式な採点配分は別途案内します。個別の理解度・点数・講評記録は公開教材とは分けて管理します。

次回への接続

第08回は電卓を題材に字句解析・構文解析・意味解析を扱います。文字列、トークン列、木と表現が変わっても、各段階の入力・出力・不変条件・検証という考え方は共通です。木の構造を読めることが、構文のまとまりの理解へつながります。

第8回　教師用補足

2025の言語導入・字句・構文・意味解析の複数回を、2026正本の電卓という題材で統合しています。歴史やCPUメーカーの説明に時間を使いすぎず、文字列→トークン→AST→評価を手元で追跡します。授業時間の分数は確定していません。

当日の中心と既存の開発経路

当日は `2+3*4` の入力位置、規則の呼出し、部分AST、返る木、評価を一続きで説明することと、失敗の段階をテストで区別することを中心にします。演習A・Bを足場にCへ進めます。トークンが読めること、文法の規則を読めること、パーサー全体を白紙から実装できることは別の段階です。

Cの二経路（完成基礎電卓の拡張と自作の全実装）を先に確認します。完成基礎電卓を利用する場合は、動く字句・構文・評価を足場に本人の独立した検査・表示・テストを増やします。starterの実行だけで拡張課題を完成したとは扱わず、本人の変更と検証を確かめます。自作の全実装経路では、字句・構文・評価を別関数として開発し、同じ受入例と完成参照で比較します。全処理系の白紙実装が授業内に完了する前提を置かず、開始時のコードと進捗を確認します。

受入例、提出物、数値文法の基本必須と発展の区別は、どちらの経路でも共通です。経路選択を課題削除や正式な配点差の根拠にしません。AST・テスト・README・reflectionから、参照の利用と本人の実装範囲を区別して説明できるかを見ます。

入力位置と木をつなぐ解説

self・current・takeを読む前提

第2回だけからクラスやselfが既習であるとは扱いません。Parserはトークン列と次に読む添字を持つオブジェクト、Number/Binaryは式の木の情報を持つオブジェクトとして、読むための役割を補います。`parser.expression()` 内のselfはそのparser自身です。全規則は同じselfを使うため、子の規則が進めたposは呼出し元でも進んでいます。クラス・dataclass・propertyの構文全体をこの回の追加課題にしません。

`current` は `tokens[pos]` を読むだけです。`take()` は進む前のcurrentを返し、posを1増やします。トークン添字posと文字列の位置offsetを混同しないようにします。`2+3*4` では偶

然一致しますが、複数桁や空白を含む入力では一致しないことがあります。eofは終わりを示すトークンで、この正常例ではpos=5のまま見て終了します。

2+3*4の帰り道

1. `parse` がParserを作り、pos=0でexpression→term→factorと呼びます。factorが2をtakeしてpos=1になり、Number(2.0)を返します。termはcurrentの+を見て、乗除算ではないため消費せず同じ木を返します。
2. expressionは左の2の木を保持し、+をtakeしてpos=2になります。右のtermを呼びます。factorが3をtakeしてpos=3になり、Number(3.0)が右のtermの左の木になります。
3. termは*をtakeしてpos=4になります。factorが4をtakeしてpos=5になり、Number(4.0)を返します。termは `Binary("*", Number(3.0), Number(4.0))` を作ります。次がeofなので、この部分ASTをexpressionへ返します。
4. expressionは保持していた2と返った乗算の木を `Binary("+", 左, 右)` につなぎます。次がeofなので返り、parseも余分なトークンがないことを確認します。ここではまだ12や14を計算していません。
5. evaluateがASTを読み、右の3×4を12、根の2+12を14と計算します。`calculate` の値は14.0で、calculatorのCLIは整数値を14と表示します。

講義のexpression断片は `op = self.take().kind` を別行に置いています。公開版の `Binary(self.take().kind, node, self.term())` でも、Pythonは引数を左から評価するため、+の消費→左の木の参照→右のtermの呼出し→Binaryの構築の順になります。返ってくる木と、評価した数値を分けて指させます。

口頭では「termはなぜ+を残すか」「currentを見ただけでposは変わるか」「右のtermが返る時点で12を計算済みか」を聞きます。答えは、加減算をexpressionが担当するため／変わらない／ASTを作っただけでまだ計算していない、となります。位置・規則・木を同時に説明できなければ、一度に全コードを読むのではなく、最初のfactorが返るところへ戻します。

確認問題の解説

1. `2`、`+`、`*`、`3` はそれぞれ正当なトークン。`+` の後にfactorが必要だが `*` が来るため構文エラー。

2. ASTは計算に必要な構造を表す。空白と括弧の表面情報は、位置情報や具体構文木を別に残す設計もあるが、今回の評価には不要。
3. Binary('-', Binary('-',10,3),2)で5。右結合なら9になり仕様が変わる。
4. 字句/構文の規則だけでは「演算対象が0」を一般に表せない。1/(2-2)もある。静的な定数畳み込みで先に発見する設計は可能だが、文法と評価条件を混同しない。

誤解への対応

意味解析と実行は全ての処理系で同じ段階になる訳ではありません。教材の電卓は数値のみなので、評価時にゼロ除算を検査します。名前と型の静的検査は後の言語へつなぐ補足です。floatの限界は数値表現の設計問題として区別し、計算結果が必ず10進の厳密値になるとは説明しません。

数値文法は、文字を消費しない再帰を使わず、digitsと繰り返しで定義しています。負数は数値トークンと一体化せず単項演算として扱うため、2-3との区別を説明できます。参考コードは大きな入力向け最適化や深さ制限を主題とせず、小入力の理解を優先します。

理解の確認

木を指して、「この子を先に計算する理由」「括弧不足を発見する関数」「テストが通っていても数値の精度に限界がある理由」を聞きます。正式な点配分は設定しません。自力実装とstarter拡張のどちらでも、段階・仕様・反例の説明を観察します。

本資料は公開補足であり、実際の学生提出物や採点データを含みません。

第9回 教員用ノート

授業の位置付け

LispのREPLと四則演算を扱う、2026年度の授業計画に沿う新作教材です。PythonホストとLisp方言は教材の設計選択です。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しません。

進行の重点

第8回の中置電卓と、第9回の前置S式を比べます。読取と評価を分け、括弧不足とゼロ除算を異なる段階の失敗として扱います。REPLは1行1式であり、複数行入力機能は今回の条件に含めません。

解答と説明

- `(+ 1 (* 2 3))` は `(* 2 3)` が 6、外側が 7。
- `(* (+ 2 3) (- 9 4))` はそれぞれ 5、最後に 25。
- `(/ 8 2)` は数値として 4。整数・浮動小数点の表示差は契約として確認する。
- 括弧不足と余分な閉じ括弧は読取の失敗。未知の演算子とゼロ除算は評価の失敗。
- REPLで `(/ 1 0)` の後に `(* 4 5)` が 20 になることが回復の証拠。

読取器は入れ子のリストを返し、評価器はそのリストの引数を再帰的に評価します。授業の木は自作図であり、原資料の図の転載ではありません。

よくある誤解

「括弧を捨ててから空白分割すればよい」では入れ子の構造が失われます。「エラーはすべて構文エラー」では対処箇所を区別できません。「先頭は常に通常の関数」では次回の特殊形式を扱えません。今回は四則に限定し、一般化し過ぎないようにします。

口頭確認

`(* 2 (+ 3 4))` を渡し、木を描いて結果 14 の理由を説明させます。閉じ括弧を1つ減らし、どの段階で何を検出するか追問します。例外をループの外で捕まえた場合、次の入力を受けられるか確認します。

到達基準案

正常な入れ子の式を計算でき、読取・評価・表示をコードで示せることを確認します。失敗入力の後にも計算でき、EOF終了を説明できることも確かめます。値だけを答えた場合は、引数を評価する順序を追問します。

支援の順序

数値1個、2引数加算、入れ子、エラー回復の順に縮めます。読取器の内部値を先に観察し、構文と演算の不具合を同時に直させません。AI利用の確認では提案を採用した反例と本人の説明を聞きます。

参照と検証

完成参照は `examples/09-lisp-arithmetic/lisp.py` です。`starter.py` は、starterが追加するsquare組込みへ負数の入力検証を加えるウォームアップ用で、本課題全体の穴埋めではありません。`exercises/09.md` の受入例と、本人の作業用コードの結果を比較します。完成参照を使った範囲を明記させます。

第10回 教員用ノート

授業の位置付け

変数と関数の定義を扱う、2026年度の授業計画に沿う新作教材です。PythonホストとLisp方言は教材の設計選択です。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しません。

進行の重点

`define` と `lambda` を分離し、通常の関数適用と特殊形式を区別します。定義時環境と呼出し時環境を、実例で繰り返し確認します。自分の版の環境構造を描いてから参照と比較させます。

授業内の中心と開始条件

当日はdefine、単純なlambda適用、非選択枝を評価しないifを中心に確認します。make-adderの環境寿命とfactの復帰は、結果だけでなく状態を説明する理解課題として扱います。第9回の自作版の進捗を確認し、名前・関数の自力実装が未完の場合は参照で状態を観察してから差分を継続します。

これは進め方の区別です。define/lambda/if/quote、クロージャ、再帰の到達点、提出仕様、受入例、成果物は演習資料のとおりとし、正式な配点はここで設定しません。全機能が1回の白紙実装で完成するとは仮定しません。正式な授業時間は未確定で、特定の分数をシラバスの指定として示しません。

make-adderの4状態

G、E5、E3とC0、C5は図の説明用ラベルです。GはホストLispの大域環境、C0はmake-adderの関数値、C5は返された内側の関数値を指します。

1. 定義時はC0が仮引数x、本文の内側lambda、Gへの参照を持ちます。xやyの具体値はまだ束縛しません。
2. make-adder 5を呼ぶと、Gを外側にしたE5へx=5を束縛します。E5で内側lambdaを評価し、仮引数y、本文(+ x y)、E5への参照を持つC5を作ります。本文の加算はまだ行いません。

3. 外側の呼出しは終了し、Gのadd5へC5を登録します。C5がE5へ参照を持つため、実行中の呼出しが終わってもx=5を探せます。xの数値だけを関数へコピーした説明にせず、環境への参照の保持を確認します。
4. add5 3を呼ぶと、保存環境E5を外側にしたE3へy=3を束縛します。yはE3、xはE5、+はGで見つかります。3と5を加え、8を返します。

「呼出しから戻れば ローカル環境は必ず消える」と説明した学生には、C5からE5への参照を指させます。呼出し元のGをE3の外側にする案ではxが見つかりません。add7を別に作れば、そのx=7は別の環境であり、add5のx=5を上書きしません。

fact 2の下降と復帰

fact 2はn=2の環境で停止条件を判定し、乗算の引数としてfact 1を呼びます。fact 1は別のn=1の環境でfact 0を呼びます。各環境の外側には、同じfactの定義を持つGがあります。呼出し中のnを1つの辞書で上書きする設計と区別します。

fact 0は1を返します。fact 1は待っていた乗算(* 1 1)を完了して1を返します。fact 2は(* 2 1)を完了して2を返します。下降のn=2,1,0と、復帰のn=0,1,2を分けて説明させます。その理解で、fact 4=24も確認します。

停止条件があるだけではすべての入力の停止を保証しません。負数の扱いは、本人の仕様と検証範囲を分けて記録させます。負数対応を新たな必須仕様として追加しません。

補足観察の使い方

`python3 examples/10-lisp-functions/learning_trace.py closure` は完成参照のInterpreterによるmake-adder呼出しとadd5呼出しを観察し、捕捉したE5と新しいE3を示します。`fact` はfact 2の下降・復帰を示します。補足driverは新しい提出物やfixtureではありません。本人の図と観察を比較します。

コードが未完でも完成参照で理解確認は可能です。参照の結果と自作コードの進捗を別に記録させます。AIの説明が「関数にxの値だけ保存」や「呼出し元を外側にする」になっていないか、図と反例で確かめさせます。

解答と説明

- `(begin (define x (+ 2 3)) x)` は `5`。名前の構文 `x` 自体を評価せず登録する。
- `square 6` は `36`。本文の評価は呼出し時まで待つ。

- `make-adder 5` が返した関数は `x = 5` の環境を保持し、`add5 3` は `8`。
- 外側の `x = 100` と仮引数 `x = 4` は別の環境。`f 4` は `5`、外側の `x` は `100`。
- `(if #f (/ 1 0) 7)` は `7`。非選択枝は評価しない。
- `(quote (+ 1 2))` はリストを返す。内部の `+` の名前検索も計算も行わない。
- `fact 0` は `1`、`fact 4` は `24`。現在の引数環境の外側で `fact` 自身を探す。

本教材の偽値は `#f` です。Pythonの真偽判定をそのまま流用すると、`0` や空リストを偽にしてしまいます。

よくある誤解

「関数は本文だけ保存する」では自由変数を解決できません。「呼出し元の環境を外側にする」はレキシカルスコープと異なります。「ifは引数を全評価する組込み」で非選択枝の失敗が発生します。「quoteは文字列を返す」ではS式データを扱えません。

口頭確認

`make-adder` の環境の寿命を聞きます。続いて `add7` を別に作り、2つの関数が保存する `x` の違いを追問します。`fact 2` の下降と復帰を描かせ、本文を定義時に評価すると何が起きるか聞きます。

到達基準案

`define` と `lambda` の役割、特殊形式の評価順序を説明できることを確認します。クロージャと名前の隠蔽の反例を通して、別環境を描けることも確かめます。再帰については、停止条件と自己参照の探索先を述べられることを求めます。可変長引数など追加機能は本人の仕様に従って確認します。

支援の順序

まず単純な変数、次に引数1個の関数、次に自由変数を持つ返却関数へ進みます。定義時と呼出し時のログへ別の印を付けさせます。AIの説明が動的スコープになっていないか、本人が反例を作ります。

参照と検証

完成参照は `examples/10-lisp-functions/lisp.py` です。`starter.py` は、starterが追加するsquare組込みへ負数の入力検証を加えるウォームアップ用で、本課題全体の穴埋めではありません。`exercises/10.md` の受入例と、本人の作業用コードの結果を比較します。完成参照を使った範囲を明記させます。

第11回 教員用ノート

授業の位置付け

スクリプト実行と評価器の準備を扱う、2026年度の授業計画に沿う新作教材です。Pythonホストと Lisp方言は教材の設計選択です。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しません。

進行の重点

式の読取を終端まで反復する入口を作り、同じ環境で評価します。字句解析では文字列内・外の状態を分けます。後半はリスト・型判定・入出力が評価器開発で何に使えるか調べます。

当日の中心と自作改修の扱い

主経路は「ファイル全体から複数式を読み、同じ大域環境で順に評価する」です。先にdefineと2回の呼出しを通してから、文字列状態・list操作へ移ります。第10回の自作版で何が既に動くかを確認し、完成ホスト上での観察と本人の改修を明示します。

文字列・コメントのlexer、list組込みまで白紙から一度に実装できる前提を置きません。不足箇所は状態例で切り分け、継続作業として本人の仕様と結果へ戻します。受入例・成果物・到達目標は演習資料のとおりとし、新しい提出条件も加えません。正式な授業時間は未確認であり、開始時の実装進捗に合わせて説明と作業の配分を調整します。

解答と説明

`lesson11.lisp` の表示は `25` と `49` です。定義した `square` を後の式が使います。スクリプトでは途中の式の値を自動表示しませんが、CLIは最後の式の値を表示します。この例は `25`、`49` に加え、末尾の `newline` の返り値 `()` を表示します。

- `car` は `(10 20)` の先頭 `10`。
- `cdr` は残りのリスト `(20)`。
- `cons 5 (10 20)` は `(5 10 20)`。
- `total (2 3 4)` は `9`、空リストは `0`。
- `"a;b"` の `;` はコメント開始ではない。

- 存在しないファイルは読取前、括弧不足は読取時、未定義名は評価時の失敗。

参照スクリプトは失敗したら停止し、非ゼロ終了します。REPLのエラー回復と同じ動作だと決めつけないようにします。本人の方針が異なる場合は文書化と結果が一致するか確認します。

途中状態の説明

共有環境

5フォームのsquare例では、defineを評価した後の大域環境をそのまま次のフォームへ渡します。2回の呼出しは別々の局所xを持ち、大域のsquare定義を共有します。大域環境の再初期化と、関数適用時の局所環境作成を同じ「新しい環境」として説明しません。参照版はファイル全体のread_allに成功してから順次評価するため、構文エラーならdefineも実行しません。評価時の失敗では、それ以前の出力・定義を取り消しません。

lexer

`(display "a (b); c") ; memo` を読み、引用符の前・文字列内・引用符の後・コメント内を指で追います。外側では括弧が構造、セミコロンがコメント開始です。文字列内ではどちらも内容です。参照版はToken.stringとSymbolで値の種類を区別し、文字列内のループでエスケープも処理します。概念図のstate名を実装にそのまま要求しません。期待トークンは`(`、Symbol(display)、文字列a (b); c、`)` です。

total

呼出しのxsは `(2 3 4)`、`(3 4)`、`(4)`、`()` と縮みます。空の呼出しが0を返した後、4+0=4、3+4=7、2+7=9と復帰します。空のxsに対してcar/cdrを呼ぶ分岐へ入らない点が停止条件の意味です。cdrとconsは新しいlistを返す教材方言なので、元のリストを破壊して縮める説明は誤りです。

学生が読める途中例は `lectures/11.md` と `exercises/11.md` にそろえています。演習の空・1要素・複数要素の検証へ戻し、別の提出用fixtureを増やしません。

よくある誤解

「ファイルは全体を1リストとして読めばよい」では独立した定義の列を誤解します。「リストはすべて関数呼出し」では引用データを扱えません。「記号も文字列もPythonのstrなの

で同じ」で名前検索とリテラルを区別できません。

口頭確認

`(list (+ 1 2))` と `(quote ((+ 1 2)))` の違いを聞きます。結果は `(3)` と `((+ 1 2))` です。ファイルで毎回環境を初期化する案の反例を作らせます。読取とOS入出力を評価器から分ける理由を聞きます。

到達基準案

複数式のスクリプトを実行し、定義を共有できることを確認します。文字列と記号を区別し、リスト再帰の停止条件を説明できることも確かめます。評価器に必要な操作を対象式と結び付けて述べられることを求めます。ホスト依存は現時点の境界として扱います。

支援の順序

2個の数値式、定義と呼出し、明示表示の順に確認します。文字列字句の不具合は空白・;・括弧を1種類ずつ試します。AIのトークン分割案は文字列の反例で確認します。

参照と検証

完成参照は `examples/11-lisp-scripts/lisp.py` です。`starter.py` は、starterが追加するsquare組込みへ負数の入力検証を加えるウォームアップ用で、本課題全体の穴埋めではありません。`exercises/11.md` の受入例と、本人の作業用コードの結果を比較します。完成参照を使った範囲を明記させます。

第12回 教員用ノート

授業の位置付け

Lisp製評価器の最小実装を扱う回です。2026年度の授業計画に沿う新作教材です。Pythonホストと Lisp方言は教材の設計選択です。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しません。

進行の重点

必要な機能を対象式から導き、Lisp製評価器を最小段階から作ります。完成ホストを使って対象評価に集中する経路も認めますが、本人の実装範囲を明記させてください。第12回のmeta参照は最小版です。対象の完全な名前・関数処理は第13〜14回へ接続します。

当日と後続を分ける進行

当日の自作対象は、演習の受入例にある数値、quote、二項四則、ifです。名前検索・define・対象クロージャは入力・結果・失敗の契約を設計し、第13回の説明と第14回の統合へつなぎます。ホストのdefine/lambdaは評価器を書くために使いますが、それを対象の機能の完成とは数えません。

第12回のm-evalはexpression、env、depthを取りますが、m-globalは空リストで、envによる名前検索はまだ実装していません。参照版の真偽値・文字列の自己評価や可変長算術を、学生課題の二項四則という契約を広げる根拠にしないでください。この区分は進行の明確化であり、受入例・提出物・機能契約・配点を変更しません。正式な授業時間を断定せず、最小例を通す作業枠を確保してください。

解答と説明

対象式 `7` は `7`、`(+ 1 (* 2 3))` は `7`。`(quote (+ 1 2))` は `(+ 1 2)`。`(if #f (/ 1 0) 7)` は `7` で、非選択枝を評価しません。

ホストに `(quote (quote (+ 1 2)))` を渡すと、ホストが外側の引用を処理して対象式 `(quote (+ 1 2))` を得ます。それをLisp製評価器が処理し、対象の結果 `(+ 1 2)` を返します。

環境の表現は連想リストでも別の形式でもかまいません。検索、定義、拡張の契約をそろえます。対象の名前をホスト環境から検索する実装は、環境の混同になります。組込みの算術

をホストへ委譲しても、対象式の分類と引数の評価規則をLispで実装したことを説明します。

二重quoteの途中状態

ロード済みホストでの呼出しは `(m-eval (quote (quote (+ 1 2))) (m-global) 0)` です。ホストが第1引数の外側quoteを処理し、m-evalのexpressionには対象式 `(quote (+ 1 2))` が渡ります。headはSymbol(quote)、argsは `((+ 1 2))` です。最小metaはquoteの引数数を確認し、car(args)でリスト値 `(+ 1 2)` を返します。ホストは返ったリストを表示するだけなので3へ進みません。

対象のquoteを含まない `(quote (+ 1 (* 2 3)))` を入力として渡した場合は、内側の乗算が6、外側の引数値が(1 6)、+のprimitiveが7を返します。引数再帰をLisp側が実装していることと、算術自体をホストへ委譲していることを分けて説明してください。

環境の表現を後続へつなぐ

説明用の連想リスト `((x 3) (y 4))` はbindingsに対応します。第14回参照は `frame-cell = (bindings)`、`env = (frame-cell parent)` です。m-bindingsはcar(car(env))で束縛表を得ます。m-lookupは自分のbindingsを検索してからparentへ進みます。m-defineは共有するframe-cellの先頭を差し替えるため、クロージャも更新後の束縛を参照できます。m-new-envとm-bindが仮引数を持つ子環境を作ります。

教材方言のcdrはコピーを返すので、束縛のcdrへ値を設定するだけでは共有環境を更新できません。この違いは第11回のlist操作と結び付けます。環境例は契約を読む橋であり、すべてを第12回に新規実装させる時間要求ではありません。学生が別の表現を選ぶ自由は保持します。

公開する図・状態表は `lectures/12.md` と `exercises/12.md` に対応しています。自作最小meta、完成最小参照、第14回完成参照のどれで観察した結果かを質問してください。

よくある誤解

「evalという名前を付ければ評価器」では仕事が分かりません。「ホストにlambdaがあるので対象にも完成」では層を混同します。「Lispで書けばPython依存が消える」では実際の起動と読取・入出力を無視しています。独立ブートストラップやネイティブ実行は成果に含めません。

口頭確認

数値と未定義記号を同じように返す実装の反例を聞いてください。次に、二重の引用がどの層で外れるか説明させてください。通常の適用でevalとapplyが互いを呼ぶ役割を、対象の関数例で追問してください。

到達基準案

数値・引用・四則・分岐の最小評価器が動き、対象とホストを区別できます。不足機能を契約として記述し、次の拡張順序を説明できます。関数の完成を一律に要求せず、第14回までの開発へつなぎます。

支援の順序

まず数値だけ、次に引用、次に二項加算を通します。名前検索やクロージャを一度に加えさせないでください。対象環境を図に描き、AIの案がホストに丸投げしていないか本人に検証させてください。

参照と検証

完成参照は `examples/12-meta-evaluator/lisp.py` です。`starter.py` は登録済みのsquare組込みへ負数の入力検証を追加するウォームアップ用で、本課題全体の穴埋めではありません。`exercises/12.md` の受入例と、本人の作業用コードの結果を比較します。完成参照を使った範囲を明記させてください。

第13回 教員用ノート

授業の位置付け

理解確認とディスカッションの回です。2026年度の授業計画に沿う新作教材です。Pythonホストと Lisp方言は教材の設計選択です。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しません。

進行の重点

理解確認をディスカッションで行います。講師の連続説明を主活動にせず、予測、ペア説明、実行、反証、説明の更新へ時間を配分してください。時間の具体的な分数は授業運用に合わせて決め、正本シラバスの指定としては記載しません。

授業内は2〜3テーマを選び、予測から説明更新まで完了させます。残る議題は予復習で検討します。演習の議題・受入条件・成果物案は変更しません。問いを順に全て解説して、学生の活動時間を消費しないでください。

第12回の自作最小metaに名前やクロージャが未実装でも参加できます。引用と#fによる分岐は自作最小meta、比較を含む分岐・関数・クロージャは対応ホストまたは完成参照、二層traceは付属の完成参照など、使う版を議論前に区別してください。付属stage12のmetaは比較演算子=も未対応なので、議題の `(if (= 1 1) 9 missing)` をstage12で当然に実行できるとは扱いません。完成参照の結果は自作版の実装達成と区別して記録してください。第14回の機能実装をこの討論回へ前倒ししないでください。

議題の解答

- `(quote missing)` は記号 `missing`。名前を検索しません。
- `(if #f (/ 1 0) 8)` は `8`。
- `(if (= 1 1) 9 missing)` は `9`。
- `plus2 7` は `9`。返却された関数が `x = 2` の環境を保持します。
- 外側の `x = 100` と引数 `x = 4` の例は `5`、`100`。共有辞書へ仮引数を書き込む設計では外側が壊れます。
- 追跡の正常結果は `7`。`[host depth=N]` と `[meta depth=N]` を区別します。metaもPython製ホスト上で実行するのでホスト側のログが現れます。

予測が誤っていても、実行後に原因を具体的に説明し修正した記録を理解確認の材料にします。最終値だけを当てた参加者には環境や非評価部分を追問してください。

よくある誤解

「ログが多ければ説明できる」では環境探索先が不明な場合があります。「動作が同じなら設計も同じ」では追加機能や反例で差が出ます。「AIが説明したので理解済み」では本人が初見入力を追えません。

口頭確認と追質問

`make-adder` の `x` がなぜ残るか聞き、異なる2つの加算器を作る入力を追問してください。`if` を通常の引数評価で実装する案を出し、反例を求めてください。Lisp製評価器からPythonホストへの依存を1個挙げさせ、除く場合の追加作業を聞いてください。

到達基準案

本人が実行前の予測と理由を書けます。他者へ問いを出し、反例またはログで検証できます。説明が変わった場合、その理由を記録できます。未解決の問いに次の検証入力があります。議論の声量や速さを理解の代用にしません。

支援の順序

発言しにくい参加者には、最初に個人メモを書く機会を設けてください。ペアの説明が一致したら、講師が新しい小さな入力を1個加えます。コードが未完成でも、完成参照と本人の版の差を使って議論できます。

参照と検証

完成参照は `examples/13-language-review/lisp.py` です。`starter.py` は登録済みのsquare組込みへ負数の入力検証を追加するウォームアップ用で、本課題全体の穴埋めではありません。`exercises/13.md` の受入例と、本人の作業用コードの結果を比較します。完成参照を使った範囲を明記させてください。

第14回 教員用ノート

授業の位置付け

Lisp製評価器とREPLを扱う回です。2026年度の授業計画に沿う新作教材です。Pythonホストと Lisp方言は教材の設計選択です。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しません。

進行の重点

第14回は最小metaから全統合への差分が大きい回です。第13回を実装回に読み替えず、開始時に数値・算術・引用・分岐と、自作の名前・関数の進捗を確認してください。当日は `square` の対象適用と、定義を保つREPLの正常・失敗・正常を中心に作業します。クロージャ・再帰の理解確認は維持し、未完の自作機能は継続して仕上げます。

これは授業内の進め方の区別であり、演習に示す提出仕様、必須到達点、受入例、成果物、配点を変更しません。正式な授業時間は未確定です。特定の分数で白紙から全実装が完成するとは保証しません。

開始状態による支援

名前と関数まで動く学生は、自作評価器で演習の受入例を確認し、REPL統合へ進みます。最小版の学生は完成参照で環境・適用の途中値を追い、作業用コピーで不足箇所と接続を実装します。完成ホスト・完成評価器を利用した範囲を本人の実装と区別します。参照実行だけで自作完成とは扱いません。

`evaluator_starter.lisp` は値の自己評価だけであり、名前・クロージャ・REPLの穴埋めではありません。`starter.py` のsquare拡張も本日の統合開始コードとは異なります。作業経路を選ぶ前に、何が動くコードを持っているか聞いてください。

対象環境の解説

完成参照の環境は `(frame-cell parent)`、セルは `(bindings)`、束縛は `(name value)` です。`m-new-env` は空のセルと外側を持つ環境を返します。`m-bindings` は現在の束縛を取り出します。`m-find` は現在にない名前を外側で探し、`m-lookup` は値を返します。

外側Gの `x = 5`、子Eの `y = 3` を例に、Eの `y` は3、Eから探した `x` はGの5となります。ホストの環境を検索する処理ではありません。`define` は右辺を対象の `m-eval` で評価してから

`m-define` で現在の環境へ登録します。

クロージャは環境への参照を保存します。`m-define` が共有セルの束縛を更新するので、その環境を捕捉した関数は後から登録した自分の定義も見つけられます。参照方言のcdrはコピーを返します。取り出したcdrを書き換えたつもりでは元の束縛は変わらないため、環境操作を関数で隠す理由をここで確認してください。

squareの解答と途中状態

1. 対象のlambdaをGで評価し、`(closure (x) ((* x x)) G)` 相当の値を作ります。Gは実際の対象環境への参照であり、文字列Gではありません。
2. `(square 6)` の先頭からクロージャ、引数から値の列 `(6)` を得ます。
3. `m-apply` が個数を確認し、保存環境Gを外側に持つE6を作ります。E6へ `x = 6` を束縛します。
4. 本文 `(* x x)` の `*` はG、各 `x` はE6で見つけます。引数値6と6を組込みへ渡し、36を返します。

Gの束縛を引数で上書きしません。呼出し元を外側に付け替えません。学生には結果36だけでなく、本文の検索先を示させてください。

クロージャと再帰の解答

対象の `make-adder 5` から作った `add5 3` は8。対象のx=5の環境が返された関数から参照され、次のy=3の環境の外側となります。第10回の4状態図を対象環境へ読み替えます。

対象の `fact 4` は24。第10回の `fact 2` の下降と復帰を先に追い、各nの独立と同じfact定義の共有を確認します。自作で未完の場合も完成参照で理解確認を行い、本人の実装との差を特定します。

REPL統合の解答

`m-repl(env)` は同じ対象環境を各入力へ渡します。`read-all` で得た式の列を `m-sequence` が順に評価し、`m-eval` と `m-apply` が対象の意味を決めます。環境は循環をまたいで保持し、入力と結果は1回の処理に属します。

`guard` の保護範囲には読取、評価、display、newlineを含めます。成功なら先頭が#t、失敗なら#fと診断を含む結果を返します。失敗時は診断し、次の入力へ進みます。表示でホスト

の数値制限に達する場合も回復します。

`(define x 5)`、`(+ x 2)`、`(/ 1 0)`、`(+ x 3)` の順に、x、7、診断、8を得ます。エラー以前の束縛や出力は取り消しません。EOFと単独の:quitで終了します。空行・コメントも参照版の扱いを確認してください。

`--meta --eval` は対象評価器のみ、`--selfhost-repl` はLisp製循環を起動します。読取はPythonのtokenize/read_all、算術・リストprimitive、入出力、guard、反復whileもホストに依存します。独立bootstrapやネイティブ実行は達成範囲に含めません。

チェックポイントの扱い

補足 `examples/14-selfhosting/checkpoints.py` は、完成参照の実装で最小式、環境、クロージャ、REPLを観察するdriverです。提出条件やfixtureを増やすものではありません。引数minimal/environment/closure/replで地点を選びます。closureのE6は同じm-new-env/m-bind/m-sequenceで分解して作る観察用環境であり、通常m-apply内部の環境を捕捉したログではありません。通常呼出しの36と比較します。本人には、通った地点、入力と結果、使った参照、変更箇所を、演習の成果物案にある実行ログへ対応付けさせてください。

評価器でsquareが失敗する場合はREPLを足さず、名前、クロージャ生成、引数値、子環境、本文の順に縮めてください。評価器が動いてから、定義継続、回復、終了を同じセッションで確かめてください。

よくある誤解

「同じ結果が出たのでLisp製評価器が動いた」では入口の証拠がありません。「対象環境は毎回初期化してよい」では定義が次の入力で使えません。「Pythonが起動に関わるだけ」では読取・入出力などの依存を過小に述べることになります。独立ブートストラップ・ネイティブ実行は達成したと表現しません。

口頭確認

対象とホストに同じ名前がある場合の検索先を聞いてください。クロージャの環境を描かせ、再帰関数が自分を見つける場所を追問してください。エラー後も残す状態と捨てる状態を聞いてください。読取器をLispへ移すなら必要な機能を検討させてください。

到達基準案

Lisp製評価器を呼ぶ実行証拠があり、対象の関数・クロージャ・再帰を説明できます。Lisp製REPLが定義を保持し、エラーから回復し、EOFで終了します。未達部分がある場合は最小の再現入力と開発順序を示せます。達成範囲とホストへの委譲を明記します。

支援の順序

最初は `--meta --eval` で評価器だけを確かめます。名前、単純関数、クロージャ、再帰を通してからREPLへ統合します。参照のLisp製REPLは1行ごとにread-allし、同じ行の複数式も扱います。複数行補完はしないので、各定義を1行にして入力します。

参照と検証

完成参照は `examples/14-selfhosting/lisp.py` です。`starter.py` は登録済みのsquare組込みへ負数の入力検証を追加するウォームアップ用で、本課題全体の穴埋めではありません。`exercises/14.md` の受入例と、本人の作業用コードの結果を比較します。完成参照を使った範囲を明記させてください。

第15回 教員用ノート

授業の位置付け

講評と総括の回です。2026年度の授業計画に沿う新作教材です。PythonホストとLisp方言は教材の設計選択です。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しません。

進行の重点

開発した実装の講評と総括を主活動にします。新機能の説明を追加する回にはしません。作者の説明、聞き手の反例、結果の観察、改善案の選択をつなぎます。正式な点数配分は本教材では作りません。

共通講評を短く行い、ペアで作者と聞き手を交代する時間を先に確保してください。全体へ共有する例は授業で選び、全員の長い全体発表を当然の前提にしないでください。第14回の未完成を一斉実装の延長で埋めず、動く範囲・制限・次の検証を説明する材料にしてください。対応機能の要件や演習の成果物案は変えず、講評の進め方を整理します。

架空例の講評

例Aは算術のみですが、対応範囲と失敗条件を明示する架空の案です。仕様と実行が一致するか、読取と評価を説明できるかを見ます。

例Bは単一の環境へ仮引数を書き込む架空の案です。外側 `x = 100`、`f` の引数 `x = 4` の反例で外側の破壊を検出します。教材の正しい仕様では `f 4` は `5`、呼出し後の `x` は `100` です。

例CはPythonホスト上のLisp製評価器です。`if` とクロージャの評価をLispで行い、読取と入出力を委譲する架空の案です。入口と関連コードを示し、達成範囲を正確に述べたか確認します。

例題ディレクトリの完成実装とfixtureは自作の教材です。講義中の架空の不具合設計案を、実際の参照コードにある欠陥として紹介しないでください。実在の提出物・学生名・成績は使いません。

よくある誤解

「機能数が多いほどよい」では検証や仕様の一致を見落とします。「READMEは後で書けばよい」では再現条件が不足します。「未完成だから説明できない」では部分達成や制限を伝

えられません。「AI利用を隠す方がよい」では学習と判断の過程を確認できません。

口頭確認

成功例の結果を予測させ、関連コードと環境を示させてください。次に聞き手が反例を1つ加えます。改善案には、利用者に起こる変化と確認する入力を追問してください。前半の探索・木・計算量がどの処理に使われたか本人のコードで説明させてください。

到達基準案

対応機能と制限を証拠で示せます。講評を事実と提案に分けて記録できます。改善案の成功条件を具体的な入力で説明できます。AIに任せた部分と本人が判断・検証した部分を振り返れます。全体総括には本人の判断が変わった具体例があります。

フィードバック案

作者が次の作業へ移れるよう、再現入力と改善箇所を短く伝えてください。個人の優劣や実在の成績比較を講評材料にしないでください。授業で決める正式な提出運用と、ここに示した成果物案を区別してください。

参照と検証

完成参照は `examples/15-showcase/lisp.py` です。`starter.py` は登録済みのsquare組込みへ負数の入力検証を追加するウォームアップ用で、本課題全体の穴埋めではありません。`exercises/15.md` の受入例と、本人の作業用コードの結果を比較します。完成参照を使った範囲を明記させてください。